Real-Time Fusion and Analysis of Phone and Council Noise Data

RUNS ON THE IPHONE

iPhone 15
Built-in microphone + GPS

Sensor Logger App
Loudness + GPS, HTTP push

PUBLISHED BY DUBLIN CITY COUNCIL

DCC noise monitors
15 monitors, LAeq

Open data API
JSON feed

HTTP GET every 5min

RUNS ON THE LAPTOP (python3 app.py starts everything below)

ngrok tunnel
public address, forwards to port 8000

Receiver (Flask, /data)
receives each POST on port 8000

Poller
every 5min, keeps noise monitors, writes new readings only

Check token
missing or invalid token is refused

Merge location + microphone
location held until next GPS reading

Build batch document
one document per 1s batch, by token

RUNS ON FIREBASE (GOOGLE CLOUD)

Cloud Firestore
collections: live_batches, open_data_readings, crowd_readings

RUNS ON THE LAPTOP (analysis)

1 Clean
drop readings below -110dBFS
and rows with no GPS reading

2 Equivalent level (Leq)
energy average of decibels

3 Calibrate
dB(A) = dBFS + C, one-off offset vs Noise 11

4-6 Live comparison
4 haversine nearest monitor, 3km range query
5 inverse-distance weighting (local), Leq (city)
6 difference in dB: phone minus monitor

7 History
Leq per session, % above 70dB(A),
by hour of day and 150m grid cell

8 Share (Task 3)
30s Leq, position rounded to ~100m,
expires after 1 hour

JSON API (Flask)
Live comparison: /api/map
History: /api/history
Council monitors: /api/stations
Refresh council data: /refresh
Shared readings: /api/share,
/api/crowd, /api/shared,
/api/clear-shared

own endpoints in the same Flask app

RUNS IN THE BROWSER

Map page
live comparison, pins, Prev / Next, refresh

Insights page
history by hour and place

Share page
share and clear readings

Browser libraries
Leaflet, Chart.js, OSM tiles

Package, external service or open data

Own program, endpoint or page

Own processing step